

Announcements

- Login, open an IDE, and Folio/browser for class
- Check grades in Folio
- Download recursion2.zip

1

Announcements

- Revel Exercises due
 - ~~Chap 17~~
- Project 9 due Friday, start of lab
 - See Program Submission requirements in Folio

2

Remaining Schedule

- Course evaluations
- Friday May 2 is last lab
- Mon May 5 is last class meeting
- Final day/time on class schedule/syllabus
- Do **NOT** use Gradescope as a calendar

3

Warm-Up Questions

- What is recursion?
- What is a recursive method?
- What are the benefits to using recursion?
- How can recursive methods be used for mathematical functions?
- How are recursive method calls handled in a call stack?
- How can problems be solved using recursion?

4

Lecture Questions

- What are recursive helper methods?
- How can selection sort be implemented recursively?
- How can binary search be implemented recursively?
- How can fractals be drawn recursively?
- How are recursion and iteration different?
- What is a tail-recursive method and why are they desirable?

5

Think Recursively

- Recursive isPalindrome() method is not efficient as it creates a new string for every recursive call:

```
public static boolean isPalindrome(String s) {
    if (s.length() <= 1) // Base case
        return true;
    else if (s.charAt(0) != s.charAt(s.length()-1))
        return false; // Base case
    else
        return isPalindrome(s.substring(1, s.length()-1));
    // substring excludes endpoint, this is 1-length()-2
}
```

6

Recursive Helper Methods

- Can use a **helper method** for efficiency:

```
public static boolean isPalindrome(String s) {
    return isPalindrome(s, 0, s.length() - 1);
}
public static boolean isPalindrome(String s, int low, int high) {
    if (high <= low) // Base case
        return true;
    else if (s.charAt(low) != s.charAt(high)) // Base case
        return false;
    else
        return isPalindrome(s, low + 1, high - 1);
}
```

7

Recursive Selection Sort

- Find the smallest number in the list and swap it with the first number
- Ignore the first number and sort the remaining elements in the list recursively

8

Recursive Selection Sort

```
public static void main(String[] args) {
    double[] list = { 2, 1, 3, 1, 2, 5, 2, -1, 0 };
    selectionSort(list);
    for (int i = 0; i < list.length; i++)
        System.out.print(list[i] + " ");
}

// call recursive selection sort
public static void selectionSort(double[] list) {
    // Sort the entire list
    selectionSort(list, 0, list.length - 1);
}
```

9

Recursive Selection Sort

```
// helper method for selection sort, uses pointers to reduce list
private static void selectionSort(double[] list, int low, int high) {
    if (low < high) {
        // Find the smallest number and its index in list(low .. high)
        int indexOfMin = low;
        double min = list[low];
        for (int i = low + 1; i <= high; i++) {
            if (list[i] < min) {
                min = list[i];
                indexOfMin = i;
            }
        }
        // Swap the smallest in list(low .. high) with list[low]
        list[indexOfMin] = list[low];
        list[low] = min;
        // Sort the remaining list(low+1 .. high)
        selectionSort(list, low + 1, high);
    }
}
```

10

Activity

- Examine the code in the recursive selection sort
- What is the base case?
- What is the general case?
- How does each call move towards a base case?
- None of the methods return an array, how does the sorting occur?

11

Recursive Binary Search

- Case 1: If the key is less than the middle element, recursively search the key in the first half of the array
- Case 2: If the key is equal to the middle element, the search ends with a match
- Case 3: If the key is greater than the middle element, recursively search the key in the second half of the array

12

Recursive Binary Search

```
public static void main(String[] args) {
    int[] list = { 3, 5, 7, 8, 12, 17, 24, 29 };
    System.out.println(recursiveBinarySearch(list, 7));
    System.out.println(recursiveBinarySearch(list, 0));
}

public static int recursiveBinarySearch(int[] list, int key) {
    int low = 0;
    int high = list.length - 1;
    return recursiveBinarySearch(list, key, low, high);
}
```

13

Recursive Binary Search

```
private static int recursiveBinarySearch(int[] list, int key, int low,
int high) {
    if (low > high) // no match
        return -low - 1;

    int mid = (low + high) / 2;
    if (key < list[mid]) // search left
        return recursiveBinarySearch(list, key, low, mid - 1);
    else if (key == list[mid]) // match found
        return mid;
    else // search right
        return recursiveBinarySearch(list, key, mid + 1, high);
}
```

14

Activity

- Examine the code in the recursive binary search
- What is the base case?
- What is the general case?
- How does each call move towards a base case?

15

Directory Size

- Preceding examples can easily be solved without using recursion
- Next section presents a problem that is difficult to solve without using recursion
- Find the size of a directory including all subfolders

16

Directory Size

- A directory may contain subdirectories and/or files
- Suppose a directory contains files and subdirectories as shown below

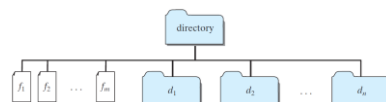


17

Directory Size

- The size of the directory can be defined recursively as follows:

$$\text{size}(d) = \text{size}(f_1) + \text{size}(f_2) + \dots + \text{size}(f_n) + \text{size}(d_1) + \text{size}(d_2) + \dots + \text{size}(d_n)$$



18

Recursive Directory Size

```
public static void main(String[] args) {
    // Prompt the user to enter a directory or a file
    System.out.print("Enter a directory or a file: ");
    Scanner input = new Scanner(System.in);
    String directory = input.nextLine();

    // Display the size
    System.out.println(getSize(new File(directory)) +
        " bytes");
}
```

19

Recursive Directory Size

```
public static long getSize(File file) {
    long size = 0; // Store the total size of all files

    if (file.isDirectory()) {
        // All files and subdirectories
        File[] files = file.listFiles();
        for (int i = 0; files != null && i < files.length; i++) {
            size += getSize(files[i]); // Recursive call
        }
    } else { // Base case
        size += file.length();
    }

    return size;
}
```

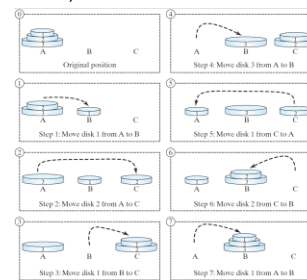
20

Tower of Hanoi

- There are n disks labeled 1, 2, 3, ..., n , and three towers labeled A, B, and C
- No disk can be on top of a smaller disk at any time
- All disks are initially placed on tower A
- Only one disk can be moved at a time, and it must be the top disk on the tower

21

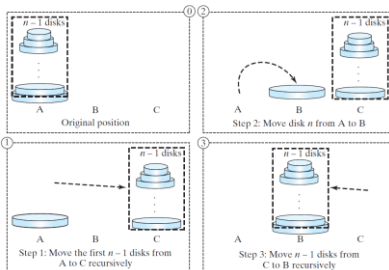
Tower of Hanoi, cont.



22

Solution to Tower of Hanoi

- The Tower of Hanoi problem can be decomposed into three subproblems



23

Solution to Tower of Hanoi

- Move the first $n - 1$ disks from A to C with the assistance of tower B
- Move disk n from A to B
- Move $n - 1$ disks from C to B with the assistance of tower A

24

Tower of Hanoi

```

public static void moveDisks(int n, char from, char to,
                             char aux) {
    if (n == 1) // Stopping condition
        System.out.println("Move disk " + n +
                             " from " + from + " to " + to);
    else {
        moveDisks(n - 1, from, aux, to);
        System.out.println("Move disk " + n +
                             " from " + from + " to " + to);
        moveDisks(n - 1, aux, to, from);
    }
}

```

25

Activity

- Examine the code in the recursive Tower of Hanoi code
- Does the code work?
- How can you verify that it works?

26

Activity

- Solve the 3-disk Tower of Hanoi problem on paper
 - Simply write down steps necessary to move all disks from one tower to the other
- Compare to results of running the program and verify they work

27

Activity

- Is there an obvious iterative solution to the Tower of Hanoi?
 - Look up iterative Tower of Hanoi solutions and compare

28

Exercise 18.3 GCD

- $\text{gcd}(2, 3) = 1$, $\text{gcd}(2, 10) = 2$, $\text{gcd}(25, 35) = 5$
- $\text{gcd}(205, 301) = 5$, $\text{gcd}(m, n)$
- Approach 1: Brute-force, start from $\min(n, m)$ down to 1, to check if a number is common divisor for both m and n , if so, it is the greatest common divisor
- Approach 2: Euclid's algorithm
- Approach 3: Recursive method

29

Approach 2: Euclid's algorithm

```

public static int euclid(int m, int n) {
    // Get absolute value of m and n;
    int t1 = Math.abs(m); int t2 = Math.abs(n);
    // r is the remainder of t1 divided by t2;
    int r = t1 % t2;
    while (r != 0) {
        t1 = t2;
        t2 = r;
        r = t1 % t2;
    }

    // When r is 0, t2 is the greatest common
    // divisor between t1 and t2
    return t2;
}

```

30

Approach 3: Recursive Method

```
public static int gcdRec(int m, int n) {
    if (m % n == 0) {
        return n;
    } else {
        return gcdRec(n, m % n);
    }
}
```

31

Fractals?

- A fractal is a geometrical figure just like triangles, circles, and rectangles, but fractals can be divided into parts, each of which is a reduced-size copy of the whole
- There are many interesting examples of fractals
- This section introduces a simple fractal, called the Sierpinski triangle, named after a famous Polish mathematician

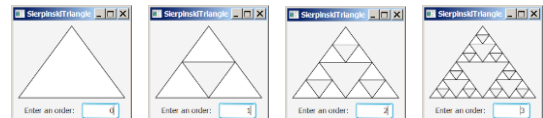
32

Sierpinski Triangle – Activity

- Begin with/draw an equilateral triangle, which is considered to be the Sierpinski fractal of order 0
- Find the midpoint of each side of the triangle
- Connect the midpoints to form another triangle (Sierpinski triangle order 1)
- Leave the center triangle intact and find/connect the midpoints of the sides of the three other triangles (Sierpinski triangle order 2)
- Repeat one more time to get an order 3 Sierpinski triangle

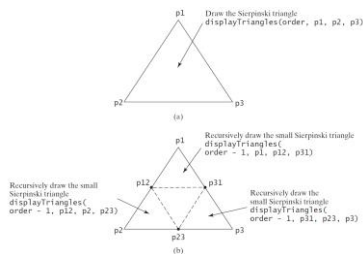
33

Sierpinski Triangle



34

Sierpinski Triangle Solution



35

Tail Recursion

- A recursive method is said to be **tail recursive** if there are no pending operations to be performed on return from a recursive call

36

Tail-Recursive Factorial

```
/** Return the factorial for a specified number */
public static long factorial(int n) {
    return factorial(n, 1); // Call auxiliary method
}

/** Auxiliary tail-recursive method for factorial */
private static long factorial(int n, int result) {
    if (n == 0)
        return result;
    else
        return factorial(n - 1, n * result);
}
```

37

Merge Sort

- Inherently recursive
- Merges two lists to sort them
- Recursively calls itself until dealing with a single item

38

Activity

- View the MergeSort.java code with a friend
- Come up with a short description of what the code is doing
- Test/Debug to visualize if necessary

39

MergeSort

- <https://www.youtube.com/watch?v=EeQ8pwjQxTM>
- Can you describe it now?

40

Topics

- Objects and Classes
 - Defining classes, constructors, reference variables (and instantiation), static, visibility modifiers, data field encapsulation, passing/returning objects to/from methods, arrays of objects, immutable objects, scope of variables, this keyword

41

Topics

- Object-Oriented Thinking
 - Class abstraction and encapsulation, class relationships, wrapper classes, boxing/unboxing, the String, StringBuilder, and StringBuffer classes
- Inheritance and Polymorphism
 - Super/sub-classes, super keyword, overriding methods, polymorphism, dynamic binding, casting objects/instanceof, equals (for objects), ArrayList, list methods, protected, final

42

Topics

- **Exception Handling and Text I/O**
 - Exception handling, finally, rethrowing exceptions, chained exceptions, custom exceptions, File class, file input/output
- **Abstract Classes and Interfaces**
 - Abstract classes, interfaces, Comparable, Cloneable

43

Topics

- **JavaFX Basics**
 - JavaFX vs Swing vs AWT, Panes, Groups, UI controls, Shapes, property binding, common Node properties/methods, Color, Font, Image/ImageView, Layout Panes and Groups, Shapes

44

Topics

- **Event-Driven Programming and Animations**
 - Events and event Sources, registering handlers and handling events, inner classes, anonymous inner class handlers, Lambda expressions, Mouse events, Key events, Observable objects, Animation
- **JavaFX Controls and Multimedia**
 - Labeled, Label, Button, CheckBox, RadioButton, TextField, TextArea, ComboBox, ListView, ScrollBar, Slider, Video and Audio

45

Topics

- **Binary I/O**
 - Text vs. Binary, binary I/O classes, Object I/O, Random Access Files

46

Topics

- **Recursion**
 - Recurrence relations, recursive methods, helper methods, recursion vs. iteration, tail recursion

47

Announcements

- **Revel Exercises due**
 - ~~Chap 17~~
- **Project 9 due Friday, start of lab**
 - See Program Submission requirements in Folio

48

Remaining Schedule

- Course evaluations
- Friday May 2 is last lab
- Mon May 5 is last class meeting
- Final day/time on class schedule/syllabus
- Do **NOT** use Gradescope as a calendar